

Reviewer Pack — Minimal Order of Elliptic Curves and Circuit Monotone Width ...

Entry 129161a9a513 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 21:23:54 UTC

Minimal Order of Elliptic Curves and Circuit Monotone Width Correlation via L-Function Arithmetic

Entry ID: 129161a9a513 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 21:23:54 UTC

1. Conjecture

Field A (mathematical branch): Elliptic Curve Theory **Field B** (complexity object): Boolean Circuits

Statement:

For every boolean circuit C with n inputs, the minimal order of an elliptic curve E associated with C (as determined by L-function arithmetic) is linearly correlated with its circuit monotone width $w_{\text{mon}}(C)$, such that $\text{ord}(E) = \Theta(w_{\text{mon}}(C))$.

Rationale (proposer's reasoning):

Elliptic curves have been used in cryptography and algebraic geometry, but their connection to circuit complexity, particularly through the lens of L-function arithmetic, is an underexplored area. This conjecture aims to expose a potential new avenue for understanding the structural properties of boolean circuits.

Taxonomy category: EllipticCurveCircuitComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 94ba77057cf969c4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the linear correlation coefficient (r) between the minimal order of elliptic curves ($\text{ord}(E)$) and circuit monotone width ($w_{\text{mon}}(C)$) for all circuits C with $n \leq 40$ inputs meets the following conditions: $r \geq 0.7$ AND $p\text{-value} \leq 0.05$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'elliptic curve' AND 'circuit monotone width' AND 'L-function arithmetic' - 'minimal order elliptic curve' AND 'Boolean circuit properties' AND 'correlation analysis' - 'L-function arithmetic in elliptic curves' AND 'circuit monotone width estimation'

Top relevant hits considered: - [http://arxiv.org/abs/1505.06503v2] Monotone orbifold Hurwitz numbers - [http://arxiv.org/abs/2506.03318v2] Validation of Quantum Elliptic Curve Point Addition Circuits - [http://arxiv.org/abs/1304.2315v2] Generalized Elliptic Integrals and Applications - [http://arxiv.org/abs/1702.06814v4] Note On Elliptic Groups of Prime Orders - [http://arxiv.org/abs/1410.4915v3] Rankin-Selberg L-functions in cyclotomic towers, III - [http://arxiv.org/abs/0706.3841v1] Arithmetic lattices and weak spectral geometry - [http://arxiv.org/abs/1704.02056v2] Counting elliptic curves with bad reduction over a prescribed set of primes - [http://arxiv.org/abs/2605.17645v2] A Local Hilbert-Pólya Realisation for Elliptic Curve L -Functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def generate_random_circuit(n):
    if n == 1:
        return ['NOT']
    else:
        gate = random.choice(['AND', 'OR'])
        inputs = [random.randint(0, len(circuit)-1) for _ in range(2)]
        circuit = [gate] + inputs
        return circuit

def compute_circuit_monotone_width(circuit):
    if not circuit:
        return 0
    elif isinstance(circuit[0], list):
        return max(compute_circuit_monotone_width(subcircuit) for subcircuit in circuit)
    else:
        return 1

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
```

```

results = []
total_instances = 0

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        circuit = generate_random_circuit(n)
        monotone_width = compute_circuit_monotone_width(circuit)

        if len(circuit) > 1: # Skip trivial circuits
            total_instances += 1

        # Simulate computing the minimal order of an elliptic curve (placeholder value)
        ord_E = random.randint(1, n * 2)

        results.append((ord_E, monotone_width))

if not results:
    return {
        "metric_name": "correlation",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

ord_E_values = [r[0] for r in results]
monotone_widths = [r[1] for r in results]

mean_ord_E = sum(ord_E_values) / len(ord_E_values)
mean_monotone_width = sum(monotone_widths) / len(monotone_widths)

covariance = sum((ord_E - mean_ord_E) * (monotone_width - mean_monotone_width) for ord_E, monotone_width in results)
variance_ord_E = sum((ord_E - mean_ord_E)**2 for ord_E in ord_E_values) / len(ord_E_values)
variance_monotone_width = sum((monotone_width - mean_monotone_width)**2 for monotone_width in monotone_widths) / len(monotone_widths)

correlation_coefficient = covariance / (variance_ord_E * variance_monotone_width**0.5)

return {
    "metric_name": "correlation",
    "metric_value": correlation_coefficient,
    "instances_tested": total_instances,
    "n_max": max(n_values),
    "conjecture_holds": abs(correlation_coefficient) >= 0.7 and p_value <= 0.05,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

```

```

    if "conjecture_holds" in trial_result and not trial_result["conjecture_holds"]:
        return {
            "metric_name": "correlation",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": f"seed={seed}"
        }

    results.append(trial_result)

mean_metric = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(r)
std_metric = (sum((r["metric_value"] - mean_metric)**2 for r in results if r["metric_value"] is not None))**0.5
support_fraction = sum(1 for r in results if "conjecture_holds" in r and r["conjecture_holds"])

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
elif any("counterexample" in r for r in results):
    first_failing_seed = next(r["seed"] for r in results if "counterexample" in r)
    print(f"RESULT: FALSIFIED counterexample=first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9aae8680.py", line 97
    return {
    ^^^^^^^
SyntaxError: 'return' outside function

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a syntax error before producing any data, which prevents us from evaluating the linear correlation coefficient and p-value. | next: Review the test code for syntax errors and ensure it runs successfully before re-evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21498
2	preregistration	ollama_remote	glm4:latest	0	0	12337
3	novelty	ollama_remote	glm4:latest	0	0	8497
4	novelty	ollama_remote	glm4:latest	0	0	9746
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17291
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	165661
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30131
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13474
9	judge	ollama_remote	glm4:latest	0	0	11909

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 290543 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/129161a9a513.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/129161a9a513.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/129161a9a513.tar.gz` (if generated)